Pathfinding on Tehran Using OSM, Traffic Simulation, DFS, and A* — Full Project Report

group members:

Vahhab Balandomanesh

Mohammadmehdi Kazemi

1. Introduction

This project focuses on implementing a complete pathfinding system on a real city map (Tehran) using OpenStreetMap (OSM) data.

The goals of the project are:

- Load and parse real map data of Tehran.
- Build a navigable road graph.
- Simulate realistic traffic conditions.
- Block specific streets (e.g., Zeinoddin Highway).
- Allow interactive route selection using draggable map markers.
- Implement and compare two algorithms:
  - Depth-First Search (DFS)
  - A* Search with a traffic-aware cost function

The final system visualizes both DFS and A* paths on interactive maps.

2. Loading and Parsing the OSM Map

We begin by using Pyrosm to load the Tehran OSM file:

```
osm = OSM(r'Tehran_2.osm.pbf')
nodes, roads = osm.get_network(nodes=True, network_type='driving')
Graph = osm.to_graph(nodes, roads, "networkx")
```

Output

A complete NetworkX MultiDiGraph representing Tehran’s road network.

Each node contains geographic coordinates,

each edge contains metadata such as:

• length

• highway type

• oneway information

• and other OSM attributes.

3. Traffic Simulation (Gaussian Model)

Real-world traffic varies with geography.

To simulate this, we apply a Gaussian-based traffic intensity model centered on

Enghelab Square.

For each road segment:

1. Compute the midpoint.

2. Compute its distance from Enghelab.

3. Apply Gaussian decay + random noise.

4. Assign traffic levels:

• high

• medium

• low

These labels later influence A* cost.

Only 1% of sampled edges are placed on the traffic map for visualization.

4. Blocking Zeinoddin Highway

To simulate road closures, the project blocks all edges connected to a specific node ID

(representing Zeinoddin):

d['access'] = 'no'

d['traffic'] = 'blocked'

When edges are blocked:

• DFS and A* both avoid them.

• A* automatically recomputes optimal detours.

5. Interactive Map for Start & Destination

We place two draggable markers:

• Start (Home)

• Destination (Parking)

Each drag automatically updates:

• The coordinates

• The nearest graph node (via osmnx.nearest_nodes)

This makes the system highly interactive and dynamic.

6. Extracting Closest Graph Nodes

Each marker's geolocation is mapped to the nearest road node:

nearest = ox.nearest_nodes(Graph, X=lon, Y=lat)

These nodes become inputs for the search algorithms.

7. DFS Pathfinding

We implement a pure DFS with:

• Stack-based search

• Blocked edge checking

• Basic path construction

DFS is simple and fast, but not optimal.

It finds any available path, ignoring distance or traffic.

We also compute the real total distance of the DFS path.

Visualization uses a red polyline.

8. A* Search with Traffic-Aware Cost

A* uses two components:

g(n):

The actual movement cost so far

(weighted by traffic).

h(n):

The geodesic (straight-line) distance between nodes

— guaranteed admissible.

Edge weights incorporate traffic:

• low → ×1

• medium → ×2

• high → ×4

Blocked edges are completely ignored.

Advantages of A*

• Always finds a valid path if one exists.

• Prioritizes fast routes.

• Avoids heavy-traffic areas.

• Produces far more realistic driving routes than DFS.

Visualization uses a lime green polyline.

9. Distance Metrics

For every A* path:

Real Distance

Sum of raw geometric lengths.

Traffic-Weighted Distance

Length × penalty (2× or 4× depending on traffic).

This metric helps evaluate:

• Time efficiency

• Traffic avoidance

• Divergence from DFS

10. Map Visualizations

The project visualizes:

• Traffic heatmap

• DFS path

• A* optimized path

• Interactive start/destination markers

Using ipyleaflet, users can visually compare both algorithms directly on Tehran's map.

11. Summary & Conclusion

This project successfully implements:

• Real-world map parsing

• Gaussian traffic simulation

• Road closures

• DFS vs. A* pathfinding

• Interactive visualization

• Real and traffic-weighted distance analysis

Key Insights

• DFS is simple but produces poor/long/chaotic paths.

• A* with a geographic heuristic is vastly superior.

• Traffic-aware weights allow the algorithm to mimic real driving behavior.

• Blocking Zeinoddin forces both algorithms to adapt their routes.

• Interactive markers make the system practical and user-friendly.

Real-World Applications

• Navigation systems

• Traffic simulation

• Urban mobility planning

• Emergency vehicle routing

• Delivery route optimization